

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

А. В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 6

ХРАНИМЫЕ ПРОЦЕДУРЫ И ФУНКЦИИ

по курсу:

МЕТОДЫ И СРЕДСТВА ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4326

подпись, дата

Г. С. Томчук
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучение создания и использования хранимых процедур и функций в MySQL для автоматизации операций с данными и реализации бизнес-логики на стороне СУБД.

2 Постановка задачи

Требуется реализовать хранимые процедуры для вставки и удаления данных с обработкой справочников, каскадного удаления, вычисления агрегатных значений и формирования статистики во временных таблицах, а также создать функции и процедуры с использованием управляющих конструкций.

Вариант: 4.

Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу.

3 Ход выполнения

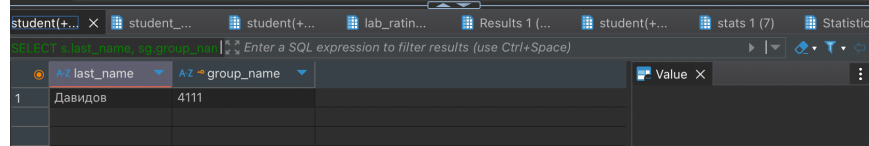
В ходе выполнения работы была использована ранее созданная база данных с информацией о студентах, группах, дисциплинах и лабораторных работах.

Была реализована процедура добавления студента с автоматическим созданием группы при ее отсутствии. На рисунке 1 приведен код процедуры вставки студента и результат ее выполнения.

```

7  -- вставка студента с автодобавлением группы
8  CREATE PROCEDURE add_student(
9      IN p_last VARCHAR(50),
10     IN p_first VARCHAR(50),
11     IN p_birth DATE,
12     IN p_group_name VARCHAR(20),
13     IN p_email VARCHAR(100)
14 )
15 BEGIN
16     IF NOT EXISTS (SELECT 1 FROM student_group WHERE group_name = p_group_name) THEN
17         INSERT INTO student_group(group_name, admission_year)
18         VALUES (p_group_name, YEAR(CURDATE()));
19     END IF;
20
21     INSERT INTO student(last_name, first_name, birth_date, group_id, email)
22     VALUES (
23         p_last,
24         p_first,
25         p_birth,
26         (SELECT group_id FROM student_group WHERE group_name = p_group_name LIMIT 1),
27         p_email
28     );
29 END //
30
31 DELIMITER ;
32
33 -- проверка вставки
34 CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
35 SELECT s.last_name, sg.group_name FROM student s
36 JOIN student_group sg ON sg.group_id = s.group_id
37 WHERE s.last_name = 'Давидов';

```



last_name	group_name
Давидов	4111

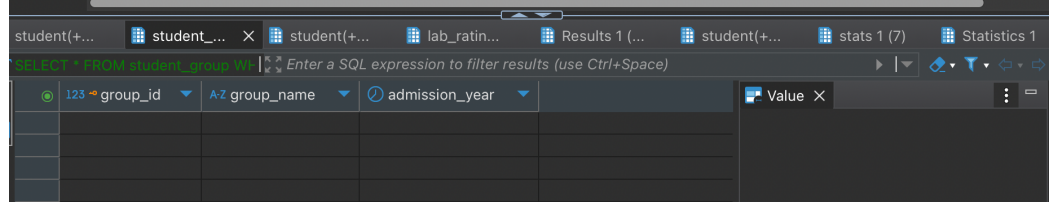
Рисунок 1 — Процедура вставки с авто-созданием группы. В результате был создан студент и группа

На рисунке 2 приведен код процедуры удаления студента с очисткой справочника групп и результат выполнения.

```

41 -- удаление студента с очисткой справочника групп
42 CREATE PROCEDURE delete_student_cleanup(IN p_student_id INT)
43 BEGIN
44     DECLARE g_id INT;
45
46     SELECT group_id INTO g_id FROM student WHERE student_id = p_student_id;
47
48     DELETE FROM student WHERE student_id = p_student_id;
49
50     IF NOT EXISTS (SELECT 1 FROM student_group WHERE group_id = g_id) THEN
51         DELETE FROM student_group WHERE group_id = g_id;
52     END IF;
53 END //
54
55 DELIMITER ;
56
57 -- проверка удаления
58 CALL delete_student_cleanup((SELECT student_id FROM student s WHERE s.last_name = 'Давидов'));
59 SELECT * FROM student_group WHERE group_name = '4111';
60

```



group_id	group_name	admission_year
----------	------------	----------------

Рисунок 2 — Процедура удаления студента вместе со всей его группой. В результате был удален студент и группа

На рисунке 3 приведен код процедуры каскадного удаления группы и связанных записей.

```
63 -- каскадное удаление группы
64 CREATE PROCEDURE delete_group_cascade(IN p_group_id INT)
65 BEGIN
66     DELETE FROM lab_rating
67     WHERE student_id IN (SELECT student_id FROM student WHERE group_id = p_group_id);
68
69     DELETE FROM student WHERE group_id = p_group_id;
70
71     DELETE FROM student_group WHERE group_id = p_group_id;
72 END //
73
74 DELIMITER ;
75
76 -- проверка каскадного удаления
77 CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
78 CALL delete_group_cascade((SELECT group_id FROM student_group sg
79 WHERE sg.group_name = '4111'));
80 SELECT * FROM student s
81 JOIN student_group sg ON sg.group_id = s.group_id
82 WHERE s.last_name = 'Давидов';
83
```

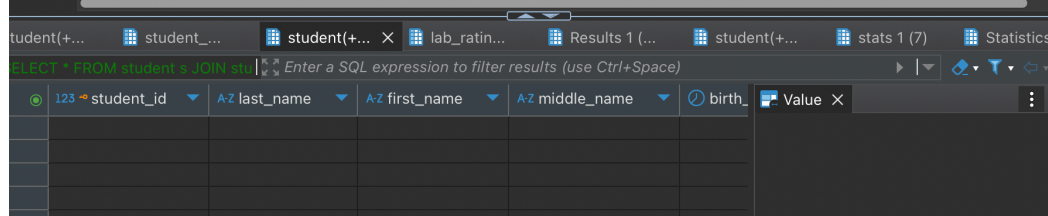


Рисунок 3 — Процедура каскадного удаления студентов вместе с группой. В результате была удалена и группа, и ее студент

Далее была реализована процедура авто-проставления оценки «0» за несданные работы по дисциплине «Базы данных», в которой используется итеративная обработка списка лабораторных работ по дисциплине. В процессе выполнения осуществляется последовательный обход записей временной таблицы и добавление отсутствующих оценок. На рисунке 4 приведен код процедуры и результат выполнения.

```

84  -- заполнение нулями отсутствующих оценок за работы (управляющие конструкции)
85  DROP PROCEDURE IF EXISTS recalc_student_status;
86  DELIMITER //
87  CREATE PROCEDURE recalc_student_status(IN p_student_id INT)
88  BEGIN
89      DECLARE i INT DEFAULT 0;
90      DECLARE lab_count INT;
91      DECLARE v_lab_id INT;
92
93      DROP TEMPORARY TABLE IF EXISTS tmp_labs;
94
95      CREATE TEMPORARY TABLE tmp_labs AS
96      SELECT lw.lab_id
97      FROM lab_work lw
98      JOIN discipline d ON lw.discipline_id = d.discipline_id
99      WHERE d.discipline_name = 'Базы данных';
100
101      SELECT COUNT(*) INTO lab_count FROM tmp_labs;
102
103      WHILE i < lab_count DO
104
105          SELECT tl.lab_id INTO v_lab_id
106          FROM tmp_labs tl
107          LIMIT 1 OFFSET i;
108
109          IF v_lab_id IS NOT NULL THEN
110
111              INSERT INTO lab_rating(student_id, lab_id, score, submission_date)
112              SELECT p_student_id, v_lab_id, 0, CURDATE()
113              WHERE NOT EXISTS (
114                  SELECT 1
115                  FROM lab_rating lr
116                  WHERE lr.student_id = p_student_id
117                        AND lr.lab_id = v_lab_id
118              );
119
120          END IF;
121
122          SET i = i + 1;
123
124      END WHILE;
125
126  END //
127
128  DELIMITER ;
129
130  -- проверка процедуры обхода лабораторных
131  CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
132  CALL recalc_student_status((SELECT student_id FROM student s WHERE s.last_name = 'Давидов'));
133  SELECT * FROM lab_rating WHERE student_id = (
134      SELECT student_id FROM student s WHERE s.last_name = 'Давидов'
135  )

```

rating_id	student_id	lab_id	score	submission_date
36	73	1	0	2026-05-10
37	73	2	0	2026-05-10
38	73	4	0	2026-05-10

Value	Description
1	Сергей Сыроежкин
3	Дмитрий Петров
5	Иван Тестов
18	Петр Петров
22	Андрей Новиков
73	Петр Давидов

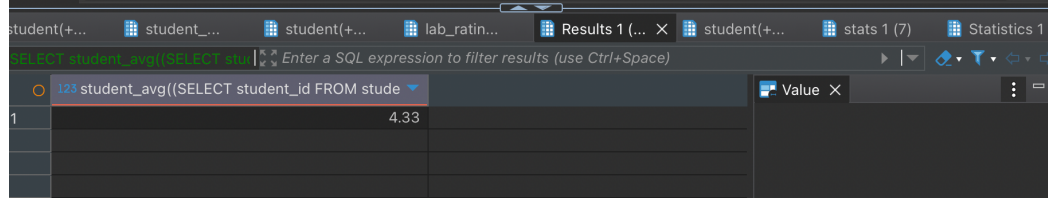
Рисунок 4 — Процедура заполнения несданных работ оценками «0». В результате были добавлены «сдачи» работ с оценкой «0»

Также была реализована функция вычисления среднего балла студента. На рисунке 5 приведен код функции расчета среднего балла и результат выполнения.

```

139 -- скалярная функция среднего балла
140 CREATE FUNCTION student_avg(p_student_id INT)
141 RETURNS DECIMAL(5,2)
142 DETERMINISTIC
143 BEGIN
144     DECLARE res DECIMAL(5,2);
145
146     SELECT AVG(score) INTO res
147     FROM lab_rating
148     WHERE student_id = p_student_id;
149
150     RETURN res;
151 END //
152
153 DELIMITER ;
154
155 -- вызов скалярной функции
156 SELECT student_avg((SELECT student_id FROM student s WHERE s.last_name = 'Сыроежкин'));
157
158

```



student_id	avg_score
1	4.33

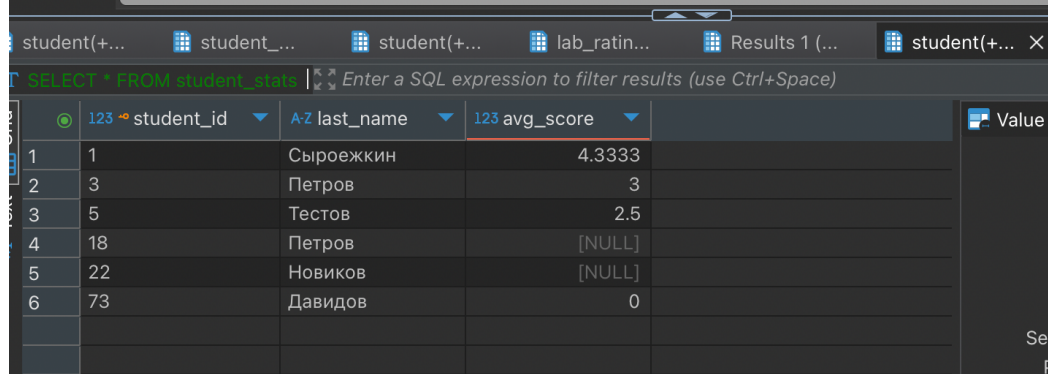
Рисунок 5 — Скалярная функция расчета среднего балла и ее результат для студента Сыроежкина

За неимением в СУБД MySQL табличных функций, для формирования агрегированной информации было создано представление (VIEW), отображающее средний балл студентов. На рисунке 6 приведен код представления и результат его выполнения.

```

158 -- табличное представление статистики студентов
159 CREATE VIEW student_stats AS
160 SELECT s.student_id, s.last_name, AVG(lr.score) AS avg_score
161 FROM student s
162 LEFT JOIN lab_rating lr ON s.student_id = lr.student_id
163 GROUP BY s.student_id, s.last_name;
164
165 -- проверка представления
166 SELECT * FROM student_stats;
167
168 DELIMITER //

```



student_id	last_name	avg_score
1	Сыроежкин	4.3333
2	Петров	3
3	Тестов	2.5
4	Петров	[NULL]
5	Новиков	[NULL]
6	Давидов	0

Рисунок 6 — Сводка со средними баллами всех студентов через View

Затем была создана процедура формирования статистики по группам с

использованием временной таблицы. На рисунке 7 приведен код процедуры формирования статистики и результат выполнения.

```
170 -- формирование статистики по группам во временной таблице
171 CREATE PROCEDURE build_stats()
172 BEGIN
173     DROP TEMPORARY TABLE IF EXISTS stats;
174
175     CREATE TEMPORARY TABLE stats AS
176     SELECT sg.group_name, COUNT(s.student_id) AS students_count
177     FROM student_group sg
178     LEFT JOIN student s ON sg.group_id = s.group_id
179     GROUP BY sg.group_name;
180
181     SELECT * FROM stats;
182 END //
183
184 DELIMITER ;
185
186 -- проверка статистики
187 CALL build_stats();
188
```

	A-Z group_name	123 students_count	Value
1	4000	4	
2	4001	1	
3	4111	1	

Рисунок 7 — Процедура подсчета количества студентов в каждой группе с использованием временной таблицы

Созданный SQL-скрипт приведен в Приложении.

4 Выводы

В ходе выполнения работы были изучены и применены хранимые процедуры и функции в реляционной базе данных. Были реализованы механизмы автоматического управления справочниками, каскадного удаления данных, вычисления агрегатных показателей и формирования статистики.

Особое внимание было уделено использованию управляющих конструкций и итеративной обработке данных, что позволило реализовать обработку набора записей внутри хранимой процедуры. Получен практический опыт переноса бизнес-логики на уровень СУБД и использования встроенных механизмов программирования SQL.

ПРИЛОЖЕНИЕ

```
USE university_db;

-- изменение разделителя команд, чтобы знак ";" интерпретировался
-- как внутренняя часть процедуры
DELIMITER //

-- вставка студента с автодобавлением группы
CREATE PROCEDURE add_student(
    IN p_last VARCHAR(50),
    IN p_first VARCHAR(50),
    IN p_birth DATE,
    IN p_group_name VARCHAR(20),
    IN p_email VARCHAR(100)
)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM student_group WHERE group_name = p_group_name) THEN
        INSERT INTO student_group(group_name, admission_year)
        VALUES (p_group_name, YEAR(CURDATE()));
    END IF;

    INSERT INTO student(last_name, first_name, birth_date, group_id, email)
    VALUES (
        p_last,
        p_first,
        p_birth,
        (SELECT group_id FROM student_group WHERE group_name = p_group_name LIMIT 1),
        p_email
    );
END //

DELIMITER ;

-- проверка вставки
CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
SELECT s.last_name, sg.group_name FROM student s
JOIN student_group sg ON sg.group_id = s.group_id
WHERE s.last_name = 'Давидов';

DELIMITER //

-- удаление студента с очисткой справочника групп
CREATE PROCEDURE delete_student_cleanup(IN p_student_id INT)
BEGIN
    DECLARE g_id INT;

    SELECT group_id INTO g_id FROM student WHERE student_id = p_student_id;

    DELETE FROM student WHERE student_id = p_student_id;

    IF NOT EXISTS (SELECT 1 FROM student WHERE group_id = g_id) THEN
        DELETE FROM student_group WHERE group_id = g_id;
    END IF;
END //

DELIMITER ;

-- проверка удаления
CALL delete_student_cleanup((SELECT student_id FROM student s WHERE s.last_name =
'Давидов'));
SELECT * FROM student_group WHERE group_name = '4111';
```



```

DELIMITER //

-- каскадное удаление группы
CREATE PROCEDURE delete_group_cascade(IN p_group_id INT)
BEGIN
    DELETE FROM lab_rating
    WHERE student_id IN (SELECT student_id FROM student WHERE group_id = p_group_id);

    DELETE FROM student WHERE group_id = p_group_id;

    DELETE FROM student_group WHERE group_id = p_group_id;
END //

DELIMITER ;

-- проверка каскадного удаления
CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
CALL delete_group_cascade((SELECT group_id FROM student_group sg
WHERE sg.group_name = '4111'));
SELECT * FROM student s
JOIN student_group sg ON sg.group_id = s.group_id
WHERE s.last_name = 'Давидов';

-- заполнение нулями отсутствующих оценок за работы (управляющие конструкции)
DROP PROCEDURE IF EXISTS recalc_student_status;
DELIMITER //
CREATE PROCEDURE recalc_student_status(IN p_student_id INT)
BEGIN
    DECLARE i INT DEFAULT 0;
    DECLARE lab_count INT;
    DECLARE v_lab_id INT;

    DROP TEMPORARY TABLE IF EXISTS tmp_labs;

    CREATE TEMPORARY TABLE tmp_labs AS
    SELECT lw.lab_id
    FROM lab_work lw
    JOIN discipline d ON lw.discipline_id = d.discipline_id
    WHERE d.discipline_name = 'Базы данных';

    SELECT COUNT(*) INTO lab_count FROM tmp_labs;

    WHILE i < lab_count DO

        SELECT t1.lab_id INTO v_lab_id
        FROM tmp_labs t1
        LIMIT 1 OFFSET i;

        IF v_lab_id IS NOT NULL THEN

            INSERT INTO lab_rating(student_id, lab_id, score, submission_date)
            SELECT p_student_id, v_lab_id, 0, CURDATE()
            WHERE NOT EXISTS (
                SELECT 1
                FROM lab_rating lr
                WHERE lr.student_id = p_student_id
                AND lr.lab_id = v_lab_id
            );

        END IF;
    END WHILE;

```

```

        SET i = i + 1;

    END WHILE;

END //

DELIMITER ;

-- проверка процедуры обхода лабораторных
CALL add_student('Давидов', 'Петр', '2002-05-10', '4111', 'davidov@mail.com');
CALL recalc_student_status((SELECT student_id FROM student s WHERE s.last_name =
'Dавидов'));
SELECT * FROM lab_rating WHERE student_id = (
    SELECT student_id FROM student s WHERE s.last_name = 'Давидов'
)

DELIMITER //

-- скалярная функция среднего балла
CREATE FUNCTION student_avg(p_student_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE res DECIMAL(5,2);

    SELECT AVG(score) INTO res
    FROM lab_rating
    WHERE student_id = p_student_id;

    RETURN res;
END //

DELIMITER ;

-- вызов скалярной функции
SELECT student_avg((SELECT student_id FROM student s WHERE s.last_name =
'Сыроежкин'));

-- табличное представление статистики студентов
CREATE VIEW student_stats AS
SELECT s.student_id, s.last_name, AVG(lr.score) AS avg_score
FROM student s
LEFT JOIN lab_rating lr ON s.student_id = lr.student_id
GROUP BY s.student_id, s.last_name;

-- проверка представления
SELECT * FROM student_stats;

DELIMITER //

-- формирование статистики по группам во временной таблице
CREATE PROCEDURE build_stats()
BEGIN
    DROP TEMPORARY TABLE IF EXISTS stats;

    CREATE TEMPORARY TABLE stats AS
    SELECT sg.group_name, COUNT(s.student_id) AS students_count
    FROM student_group sg
    LEFT JOIN student s ON sg.group_id = s.group_id
    GROUP BY sg.group_name;

    SELECT * FROM stats;

```

```
END //

DELIMITER ;

-- проверка статистики
CALL build_stats();
```